DocMind

RAG AI Document Assistant

The Complete Master Guide

From Scratch to Deployment - explained simply, with examples

Live demo: docmind-x.streamlit.app

Code: github.com/vinith-bonila/rag-doc-assistant

Author: Vinith Bonila

0. How to Read This Guide

This guide explains **everything** about the DocMind project - what it is, how every part works, why each tool was chosen, the problems we hit and how we solved them, how to run and edit it, and how to talk about it in an interview.

It is written in plain language, as if explaining to a curious beginner. Two kinds of coloured boxes appear throughout:

- **IN SIMPLE TERMS** (blue) - a real-world analogy so the idea sticks.
- **WHY WE USED IT** (green) - the reason we picked that tool or approach.

If you only have five minutes, read Sections 1, 3 and 10. If you are preparing for an interview, focus on Sections 4, 8 and 11.

---

1. What Did We Build?

DocMind is a website where you upload a document - a report, resume, research paper, manual, contract, or even source code - and then ask questions about it in normal English. It replies with an answer that is taken *only* from your document, shows you the exact page the answer came from, types the answer out live, and then adds a simple explanation to help you understand it.

> **IN SIMPLE TERMS**
>
> Imagine a very fast friend who can read a 100-page report in two seconds. You ask, 'What are the risks in this report?' and the friend not only tells you, but also points to the exact sentence and page where they read it - so you can trust the answer instead of just believing it.

You can ask it to summarise, explain, compare sections, translate, find action items, list risks, make interview questions, explain code, and more - all without any buttons or special commands. You just type what you want.

2. The Problem We Are Solving

AI language models (like the one behind ChatGPT) are trained on lots of text, but they have two weaknesses: (1) they only know what they saw during training, so they are blind to *your* private document, and (2) when unsure, they sometimes **make things up** confidently. That making-things-up is called **hallucination**.

> **IN SIMPLE TERMS**
>
> Think of a closed-book exam versus an open-book exam. A plain AI is a student taking a closed-book exam from memory - fast, but it may misremember. DocMind turns it into an **open-book exam**: before answering, the AI is handed the exact pages it needs, and is told 'answer only from these pages, and if the answer is not here, say so.'

This technique - fetch the right pages first, then answer using them - is called **RAG (Retrieval-Augmented Generation)**. It is the core idea of the whole project.

---

3. The Big Picture (The Assembly Line)

DocMind works like a factory assembly line. Your question and your document go in one end, and a trustworthy, cited answer comes out the other. Here are the stations, in order:

#	Station	What happens in one line
1	Upload & Read	Turn your PDF/Word file into plain text (keeping page numbers).
2	Chunk	Cut the text into small, meaningful pieces.
3	Embed	Turn each piece into a list of numbers that captures its meaning.
4	Store (FAISS)	Save those number-lists in a searchable index.
5	Retrieve	Find the pieces most related to your question (two ways at once).
6	Rerank	Carefully re-check the shortlist to put the best piece on top.
7	Generate	The AI writes an answer using ONLY those pieces, with citations.
8	Evaluate	A report card that scores how good the retrieval and answers are.

Stations 1-4 happen when you upload. Stations 5-7 happen every time you ask a question. Station 8 is a dashboard you can run any time to prove quality.

4. Every Step, Explained

4.1 Upload & Read the document (parsing)

Computers cannot 'read' a PDF like we do - a PDF is really a set of drawing instructions. So first we extract the plain text, page by page, and remember which page each piece of text came from (so we can cite it later).

WHY WE USED IT

We used **pypdf** for PDFs and **python-docx** for Word files because they are lightweight, free, and pure-Python (no heavy system installs) - important for a small free server. We keep page numbers so every answer can point to an exact page.

Supported formats: PDF, Word (DOCX), plain text (TXT), and Markdown (MD).

4.2 Chunking (cutting the text into pieces)

We cut the document into small overlapping pieces of about 800 characters. Too big and a piece mixes many topics; too small and it loses context. The overlap means a sentence at a boundary is not cut in half.

IN SIMPLE TERMS

Like cutting a long movie into short scenes so you can jump straight to the scene you care about, instead of re-watching the whole film. The tiny overlap is like each scene starting a few seconds before the last one ended, so nothing important gets lost between cuts.

4.3 Embeddings (turning words into numbers)

An **embedding model** reads a piece of text and outputs a list of ~384 numbers that represents its *meaning*. Texts with similar meaning get similar number-lists, even if they use different words.

IN SIMPLE TERMS

Imagine every sentence gets placed as a dot on a giant map. Sentences about 'money' land near each other; sentences about 'oil wells' land in a different neighbourhood. To find related text, we just look for nearby dots. 'Car' and 'automobile' land right next to each other even though they share no letters.

WHY WE USED IT

We used **sentence-transformers all-MiniLM-L6-v2** - a small (~90 MB), fast, well-respected model. Small matters because our free server only has about 1 GB of memory. It is strong enough for great retrieval without needing an expensive GPU.

4.4 FAISS (the searchable index)

FAISS (by Meta) stores all those number-lists and can instantly find the ones closest to your question's number-list.

IN SIMPLE TERMS

It is like the index at the back of a textbook, but a super-powered one that understands meaning. Instead of flipping every page, you ask 'what is near this idea?' and it hands you the right pages at once.

WHY WE USED IT

We used **FAISS** because it is free, extremely fast, runs on a normal computer (no cloud database bill), and is the industry-standard choice for this scale. We use its simplest exact index because our documents are small enough that exact search is instant.

4.5 Two kinds of search at once (Hybrid Retrieval)

There are two ways to find relevant text, and each is good at something different:

- **Dense search** (the embeddings above) understands *meaning* - great for synonyms and paraphrases, but it can miss an exact keyword or code.
- **Keyword search (BM25)** matches *exact words* - great for names, IDs, and acronyms like 'CPF', but it is blind to synonyms.

We run **both** and combine their results. This is called **hybrid retrieval**.

IN SIMPLE TERMS

Two friends help you find a book. One remembers what books are *about* (meaning); the other remembers *exact titles* (keywords). Asking both and merging their suggestions finds the right book far more often than asking either one alone.

WHY WE USED IT

Most beginner projects use only dense search. Using **both** (dense + BM25) is what makes retrieval genuinely reliable - it is a key differentiator and a great interview talking point.

4.6 RRF Fusion (combining the two rankings fairly)

The two searches each return a ranked list, but their scores are on completely different scales, so we cannot just add them. Instead we use **Reciprocal Rank Fusion (RRF)**, which only looks at *position* (1st, 2nd, 3rd...), not raw scores.

IN SIMPLE TERMS

Like combining two judges' rankings in a talent show. One judge scores out of 10, the other out of 100 - you cannot add those. So instead you say 'being ranked 1st is worth a fixed number of points from either judge.' A contestant both judges rank highly wins - fair and simple.

4.7 Cross-Encoder Reranking (the careful second reader)

The hybrid step is fast but rough. So we take its top ~20 candidates and pass them through a **cross-encoder** - a slower, smarter model that reads your question and each passage *together* and gives an accurate relevance score. The best few then go to the AI.

IN SIMPLE TERMS

The first search is a librarian quickly grabbing 20 books that look relevant from the shelf. The reranker is an expert who actually opens each of those 20 and reads a bit, then hands you the 5 that truly answer your question. Fast rough pass, then careful precise pass.

WHY WE USED IT

We used the **ms-marco-MiniLM-L-6-v2** cross-encoder. This retrieve-many-then-rerank pattern is the heart of a high-quality RAG system, and the reranker's score is what you see next to each citation.

4.8 Generation (the writer who only uses the notes)

Now the AI writes the answer - but it is given strict rules: use ONLY the retrieved passages, cite them like [1], [2], and if the passages do not contain the answer, say 'the document does not contain enough information' rather than guessing.

WHY WE USED IT

We used **Groq** running **Llama-3.1-8b-instant** because Groq has a generous **free** tier and is extremely fast (important for live typing). The code is OpenAI-compatible, so we could swap to OpenAI or any other provider by changing one setting - no rewrite.

This grounding + citation rule is the main defence against hallucination: every sentence can be traced back to a real place in your document.

4.9 Streaming (typing the answer live)

Instead of waiting for the whole answer, the words appear as they are generated, like someone typing in real time.

IN SIMPLE TERMS

Like a live chat where you see the reply appear word-by-word, instead of staring at a blank screen wondering if it froze. It just feels faster and more alive.

4.10 Evaluation (the report card)

How do we *prove* the system is good, not just claim it? We built an evaluation dashboard that scores four things on a set of test questions:

Metric	In plain words	Score
recall@5	Did the correct page make it into the top 5 results?	1.00
MRR	How near the top was the first correct page?	0.96
faithfulness	Is the answer actually backed by the document (not made up)?	0.78
answer relevance	Does the answer actually address the question?	0.82

WHY WE USED IT

Most student projects skip evaluation. Having real numbers that separate **search problems** from **writing problems** is what makes this look professional. If an answer is ever wrong, these metrics tell you *which* part to fix.

5. The Assistant Brain (Teaching Layer)

On top of the search engine, we added the behaviour that makes it feel like a helpful assistant (similar to NotebookLM or ChatGPT file uploads):

Structured answers

Every reply has clearly separated parts, so facts and opinions never get mixed:

- **According to the document** - the grounded, cited answer (only from your file).
- **AI explanation** - a simple explanation in easy words to help you understand.
- **Example** - a real-world analogy when it helps.
- **Sources** - clickable citations showing the exact passage and page.

Conversation memory

You can ask follow-ups. If you ask 'Explain CPF' and then 'Why is it important?', the system understands that 'it' means CPF.

WHY WE USED IT

A follow-up like 'why is it important?' has no keywords to search with. So we first **rewrite** it into a full standalone question ('why is a Central Processing Facility important?') using the chat history, then search. This is a real technique used in production chat systems.

Document scoping & never lying

Once you upload your own file, answers are limited to *your* document (so 'summarise this' summarises the right thing). And if your document truly does not contain the answer, it refuses instead of inventing one.

---

6. Why Each Technology? (The Toolbox)

Every tool was chosen for a reason - usually 'free, small, fast, and reliable', because the whole thing had to run on a free 1 GB server.

Tool	Job	Why this one (in simple terms)
Python	The language	Easiest language for AI; every library we need exists in it.
Streamlit	The website UI	Turns Python into a web app in minutes; free hosting; no need to learn front-end.
FastAPI	The backend API	A clean, fast way to expose the engine as a web service; kept for real-backend credibility.
FAISS	Vector search	Free, super-fast meaning-search that runs on a normal PC.
sentence-transformers	Embeddings + reranker	Small, free, high-quality models that fit in 1 GB memory.
rank-bm25	Keyword search	Simple, pure-Python exact-word search to pair with meaning-search.

Tool	Job	Why this one (in simple terms)
Groq + Llama 3.1	The writer (LLM)	Free tier, very fast typing, OpenAI-compatible so easy to swap.
pypdf / python-docx	Read files	Light, free readers for PDF and Word.
Docker	Packaging	Bundles everything so it runs the same anywhere.
GitHub Actions	Auto-testing (CI)	Runs the tests automatically on every change to catch breakage.
pytest	Tests	Proves the important parts work and keep working (24 tests).

7. How We Built It (The Journey)

We built it in phases, always keeping a working version at each step:

Phase	What we added
1. Ingestion	Read PDF/DOCX/TXT/MD, chunk with overlap, keep page numbers, build the FAISS index.
2. Retrieval	Dense + BM25 hybrid search with RRF fusion, plus cross-encoder reranking.
3. Generation	Grounded, cited, streaming answers; live document upload.
4. Evaluation	recall@k, MRR, faithfulness, answer relevance + a dashboard.
5. UI & Ship	Streamlit interface, Docker, automated tests, CI, documentation.
6. Assistant	Structured answers, conversation memory, document scoping.

---

8. Problems We Hit & How We Solved Them

This is the most useful section for interviews - it shows real engineering judgement, not just following a tutorial.

Deployment: finding a truly free home

- **Hugging Face Spaces** - planned, but it moved Docker apps behind a paid plan.
- **Google Cloud Run** - considered, but it needs a billing account (card on file).
- **Render free tier** - rejected: only 512 MB memory, which would crash when loading our AI models (~600-900 MB).
- **Streamlit Community Cloud** - the winner: free, ~1 GB memory, built for exactly this. We restructured the app to run everything in one process to fit.

The 2.5 GB accident

Installing PyTorch (needed by the AI models) by default downloads a huge ~2.5 GB GPU version, which broke the free build. We forced the small CPU-only version.

> **IN SIMPLE TERMS**
>
> Like ordering a bicycle and being shipped an entire truck by mistake. We told the shop 'just the bicycle, please' - the small CPU version.

The invisible library crash

FAISS needs a small system library (libgomp) that the slim server image did not include, so it crashed on start. We added that one library to the setup.

Memory that kept growing

Every time a user re-uploaded a file, its text was *added again*, so memory grew forever. We changed uploads to **replace** the old copy instead of stacking duplicates, and capped the chat history length.

IN SIMPLE TERMS

Like a notebook where re-saving the same page kept appending new copies until the notebook burst. Now re-saving overwrites the old page.

Garbled PDF text (broken characters)

Some PDFs use fonts with a broken character map, so a word like 'operations' came out with wrong symbols in the middle (for example 'opera?ons', where the 'ti' turned into a strange letter). We added a cleanup step that repairs these broken characters after reading. (Even switching PDF libraries did not fix it - the fault is inside the PDF, so cleanup is the right fix.)

Summarising the WRONG document

A user's uploaded report was mixed into the same index as our demo files, so 'summarise this document' summarised the demo instead. We added **document scoping** so answers stay inside the user's own uploaded files.

The 'stale cache' error after deploy

After some updates the live app briefly errored because it kept an old copy of the engine in memory. The fix is a one-click 'Reboot app', which clears it. Good to know: it only happens when we change how a saved object is called.

9. How to Run & Edit It (For Future You)

Run it on your computer

```
pip install -r requirements.txt
cp .env.example .env        # then put your Groq key inside
streamlit run ui/streamlit_app.py
```

That is it - the app opens in your browser and builds the sample index on first run. To run the optional API: `uvicorn api.main:app`.

Where each thing lives (the map)

File	What it controls
config.py	All the settings (models, chunk size, how many results). Start here to tweak.
src/ingest.py	Reading files + cleaning text + chunking.
src/embeddings.py	The meaning-to-numbers model.
src/vectorstore.py	The FAISS index (save/load/search/rebuild).
src/retrieval.py	Dense + BM25 + RRF fusion + document scoping.
src/reranker.py	The careful second-reader model.
src/llm.py	The answer-writing prompts, memory, and streaming.
src/pipeline.py	Ties it all together (retrieve -> rerank -> generate).
ui/streamlit_app.py	The website: upload, chat, citations, dashboard.
eval/evaluate.py	The four quality metrics.
tests/	24 automated tests that prove things work.

To change common things: want more results per answer? change `RERANK_TOP_N` in config.py. Want a different AI model? change `LLM_MODEL`. Want a different embedding model? change `EMBED_MODEL`. Everything is one setting away.

---

10. Results (What We Achieved)

- A **live, free, public** AI document assistant anyone can use.
- Works on **any** document type and any question, in plain English.
- **Grounded & cited** answers with exact page numbers - it does not lie.
- **Streaming** answers with **conversation memory** for follow-ups.
- A real **evaluation dashboard**: recall@5 1.00, MRR 0.96, faithfulness 0.78.
- **24 automated tests**, Docker packaging, and automatic testing on every change.

---

11. Interview Cheat-Sheet

Q: What is RAG in one sentence?

A: Fetch the most relevant passages from a document, then let the AI answer using only those passages - so it stays accurate and can cite sources.

Q: Why hybrid search instead of just embeddings?
A: Embeddings capture meaning but can miss exact keywords, IDs and acronyms; BM25 catches exact words but misses synonyms. Using both and fusing them recovers the strengths of each.

Q: What does the cross-encoder reranker add?
A: Embedding search scores query and passage separately (fast, rough). The cross-encoder reads them together for a much more accurate relevance score, so we retrieve many cheaply then rerank a small shortlist precisely.

Q: What is faithfulness and why does it matter?
A: It measures whether the answer's claims are actually supported by the retrieved text - i.e. how much it is NOT hallucinating. It separates writing quality from search quality.

Q: How do you stop hallucination?
A: The prompt forces the AI to answer only from the retrieved passages, cite them, and refuse when the answer is not present; plus a guard that stops it adding unsolicited outside knowledge on a refusal.

Q: How did you fit it in 1 GB of free memory?
A: Small MiniLM-class models, CPU-only PyTorch, loading models once and reusing them (lazy + cached), and making re-uploads replace instead of duplicate.

Q: How would you scale it to millions of documents?
A: Swap FAISS's exact index for an approximate one (HNSW/IVF), move the vector store to a managed service, and add persistent storage for uploads - the retriever interface is already pluggable.

12. Glossary (Simple Definitions)

Term	Meaning in plain words
LLM	A large AI model that writes text (the 'brain' that answers).
RAG	Give the AI the right pages first, then let it answer from them.
Embedding	A list of numbers that captures the meaning of a piece of text.
Vector / vector DB	The number-list, and the database that searches them by meaning.
FAISS	The fast library that stores and searches those number-lists.
Chunk	A small piece of the document (about a paragraph).
BM25	Classic exact-keyword search.
Hybrid retrieval	Using meaning-search and keyword-search together.
RRF	A fair way to merge two ranked lists using position, not raw score.
Cross-encoder / reranker	A careful model that re-scores the shortlist for accuracy.
Grounding	Forcing the answer to come only from the document.
Citation	A pointer to the exact source passage and page.
Hallucination	When AI confidently makes something up.
Streaming	Showing the answer word-by-word as it is written.
Faithfulness	How well the answer is backed by the source text.
Lazy loading	Loading a model only when it is first needed, to save memory.

End of guide. You now understand what DocMind is, how every piece works, why each tool was chosen, how to run and edit it, and how to explain it with confidence.